

Embedded Xinu

Raspberry Pi 5 (BCM2712)

ユーザーズマニュアル

Xinu-rpi5 プロジェクト

2026 年 6 月

目次

1	はじめに	2
2	システム構成	2
2.1	主要諸元	2
2.2	コンソール	2
3	カーネルのフラッシュと起動	3
3.1	ビルド	3
3.2	USB メモリへの書き込み (Mac)	3
4	4 つの OS バリエーションと kexec による切り替え	3
5	microSD ストレージ	4
5.1	一覧と読み出し	4
5.2	書き込み (永続)	4
5.3	診断	4
6	kexec カーネルセクタ	4
7	シェルコマンド	5
8	ネットワーク経由のリモート操作	5
9	WiFi	6
9.1	アクセスポイントへの接続	6
9.2	メッシュネットワーク (MANET ad-hoc)	6
10	USB (マウス・キーボード)	7

11	付録	7
11.1	ビルドターゲット一覧	7
11.2	トラブルシューティング	7
11.3	安全上の注意	7

1 はじめに

本書は、Raspberry Pi 5 (BCM2712 / Cortex-A76) 上でベアメタル動作する Embedded Xinu カーネル (xinu-rpi5) の使い方をまとめたユーザーズマニュアルである。本システムは MMU を有効化した AArch64 のシングルイメージカーネルで、以下を備える。

- HDMI フレームバッファとウィンドウシステム (デスクトップ)
- RP1 経由の有線イーサネット (TCP/IP・HTTP ゲートウェイ)
- 内蔵 CYW43455 による WiFi (スキャン / WPA2 / DHCP / ping)
- RP1 xHCI による USB マウス・キーボード
- microSD カードの読み書き (FAT32) と /microsd マウント
- RAM 上で別カーネルへ切り替える kexec セレクタ
- 1 つのソースから派生する 4 種類の OS バリエーション

■**重要な前提 (起動メディア)** . 本機は **USB メモリから起動**し、本体の microSD スロットは通常空である。USB メモリの FAT パーティション (bootfs) に置いた `kernel_2712.img` がファームウェアによって読み込まれて起動する。microSD スロットは、本カーネルの SD ドライバが扱うデータ領域として利用する (/microsd)。

2 システム構成

2.1 主要諸元

項目	値
SoC	BCM2712 (Cortex-A76, AArch64)
カーネルイメージ	<code>kernel_2712.img</code> (エントリ 0x80000)
MMU	有効 (恒等マップ、キャッシュ OFF)
デバッグ UART	0x107D001000 (3 ピン JST、115200 8N1)
HDMI	既定 1920×1080×32 (OS3 は 1280×720)
有線 IP (静的)	192.168.3.101
microSD コントローラ	sdhci-brcmstb sdio1 (0x10_00FFF000)
USB ホスト	RP1 内 DWC3/xHCI (PCIe 経由)

2.2 コンソール

出力は HDMI のテキストコンソールとデバッグ UART の両方に出る (`uart_putc` が画面にもミラーされる)。デスクトップ起動時はウィンドウシステムが画面を占有するため、テキストコンソールは隠れる。ネットワーク経由のリモート操作 (第 8 節) が最も確実な操作手段である。

3 カーネルのフラッシュと起動

3.1 ビルド

compile/ ディレクトリで make を実行する。

```
cd compile
make pi5          # フル OS          -> kernel_2712.img
make pi5-osmin    # OS1 (最小)       -> kernel_min.img
make pi5-os2      # OS2 (WMなし)     -> kernel_os2.img
make pi5-os3      # OS3 (低解像度)   -> kernel_os3.img
```

3.2 USB メモリへの書き込み (Mac)

USB メモリ (FAT パーティション = bootfs) を Mac に挿し、イメージをコピーして取り外す。マウントポイントを必ず確認すること。

```
diskutil mount /dev/disk4s1
cp kernel_2712.img /Volumes/bootfs/kernel_2712.img
sync
diskutil unmount /dev/disk4s1
```

書き込んだら USB メモリを Pi 5 に戻し、電源を入れる。ファームウェアの初期化に十数秒かかった後、起動ログが UART / HDMI に表示される。

4 4 つの OS バリエーションと kexec による切り替え

同一の main.c からコンパイルフラグだけを変えて、4 種類のカーネルを生成できる。いずれも kexec (第 6 節) で相互に切り替え可能。

名称	イメージ	特徴
フル OS	kernel_2712.img	ウィンドウシステム + ネットワーク (全機能)
OS1	OS1.IMG	シェルのみ。ネットワークなし・ウィンドウなし
OS2	OS2.IMG	全機能だがウィンドウシステムなし (テキストコンソール)
OS3	OS3.IMG	全機能・HDMI を 1 ランク下げた 1280×720

- **OS1 (最小)** : ネットワークもウィンドウも持たない。HDMI テキストコンソールに xinu-min\$ プロンプトを表示し、USB キーボードで操作する。
- **OS2 (WM なし)** : ネットワーク等の全機能を持つが、ウィンドウマネージャの代わりに全画面テキストコンソールでシェルを動かす (プロンプト xinu-os2\$)。ネットワーク経由の操作も可能。
- **OS3 (低解像度)** : フル OS と同じだが HDMI を 1280×720 に下げた版。

■読みやすさ. テキストコンソールのフォントは 8×8 を 3 倍に拡大して表示する (OS1・OS2 で有効)。

5 microSD ストレージ

本体の microSD スロットは /microsd としてマウントされる (カード未挿入でもディレクトリは存在する)。FAT32 カードの読み出しと、小サイズファイルの永続書き込みに対応する。

5.1 一覧と読み出し

```
ls /microsd          # カード上のファイル一覧
cat /microsd/CONFIG.TXT # ファイル内容の表示
```

5.2 書き込み (永続)

HTTP 経由で 1 クラスタ (32KB) 以下の小ファイルをカードへ永続書き込みできる。ボディが内容、URL 末尾がファイル名 (8.3 形式)。

```
curl -X POST --data-binary "hello from xinu" \
    http://192.168.3.101/microsd/write/TEST.TXT
# 再起動後も ls /microsd / cat /microsd/TEST.TXT で残存
```

5.3 診断

SD コントローラとカードの初期化状況を表示する診断コマンド。

```
sdtest          # コントローラ / CMD0, CMD8, ACMD41 等の応答を表示
```

6 kexec カーネルセレクト

kexec は microSD 上のカーネルイメージを RAM に読み込み、電源を切らずにそのカーネルへジャンプする (再フラッシュ不要)。失敗しても RAM 上の操作なので、電源再投入で USB のフル OS に戻るだけでブリック リスクはない。

```
kexec /microsd/OS1.IMG      # OS1 (最小) を起動
kexec /microsd/OS2.IMG      # OS2 (WMなし) を起動
kexec /microsd/OS3.IMG      # OS3 (低解像度) を起動
kexec /microsd/KERNEL~1.IMG # カード上のフル OS へ
```

■注意点.

- kexec の後はネットワークが落ちる (非ファームウェア状態からの PCIe 再初期化のため)。結果は HDMI 画面で確認する。
- ブートできるのはベアメタルの Xinu カーネルのみ。Linux カーネル (例 KERNEL8.IMG、約

9.6 MB) は別のブートプロトコルを必要とし、単純なチェーンロードではハングする。本実装は 4 MB 以上のイメージを Linux と見なして拒否する (Xinu と Linux はどちらも arm64 Image でマジックが同じため、サイズで判別する)。

7 シェルコマンド

画面のシェル (USB キーボード) か、ネットワーク経由 (第 8 節) の `/run?cmd=...` から実行する。主なコマンドを示す。

コマンド	説明
<code>help</code>	コマンド一覧
<code>ls [path]</code>	ディレクトリ一覧
<code>cat <file></code>	ファイル内容表示
<code>cd / pwd</code>	カレントディレクトリ移動/表示
<code>kexec /microsd/<k.img></code>	別カーネルを起動 (第 6 節)
<code>cc <file> / make <file></code>	C/AIPL プログラムをその場でコンパイル・実行
<code>mem</code>	ヒープ/ BSS の状況
<code>peek <hex></code>	32bit MMIO ワードの読み出し
<code>uptime</code>	汎用タイマ値
<code>ps</code>	コア/ EL 状態
<code>wifi on/off/status/scan</code>	接続/切断/状態/スキャン (第 9 節)
<code>wifi adhoc <ssid> [ch] [n]</code>	MANET ad-hoc (メッシュ) セルに参加 (第 9 節)
<code>wifi aodv <a.b.c.d></code>	マルチホップのメッシュ経路を探索 (第 9 節)
<code>sdtest</code>	SD コントローラ診断
<code>reboot</code>	再起動 (PM ウォッチドッグ)
<code>clear</code>	画面クリア

8 ネットワーク経由のリモート操作

フル OS・OS2・OS3 は有線 IP 192.168.3.101 で HTTP を待ち受ける。シリアル入力を使いにくい環境でも、Mac 等から `curl` で操作できる。

エンドポイント	説明
<code>/run?cmd=<shell></code>	任意のシェルコマンドを実行し出力を返す
<code>/fs, /fs/ls/<d>, /fs/cat/<f></code>	VFS ツリーの参照
<code>/fs/write/<f> (POST)</code>	VFS (tmpfs) ヘファイル書き込み
<code>/microsd/write/<NAME> (POST)</code>	microSD へ永続書き込み (第 5 節)
<code>/usb/...</code>	USB の列挙・診断 (第 10 節)
<code>/api/actors, /send?to=&m=</code>	アクタ/ AIPL ゲートウェイ

```
curl 'http://192.168.3.101/run?cmd=ls%20/microsd'
curl 'http://192.168.3.101/run?cmd=cat%20/microsd/CONFIG.TXT'
curl 'http://192.168.3.101/run?cmd=wifi%20status'
```

補足：URL 中の空白は %20 もしくは + でエンコードする。出力バッファには上限があるため、長い結果は分割される。

9 WiFi

内蔵 CYW43455 によるフル WiFi（スキャン／WPA2／DHCP／ping）に対応する。起動時には自動接続しない。wifi on を実行したときのみ接続する。

9.1 アクセスポイントへの接続

```
wifi on <ssid> <pass> # ファームウェア起動(~10s) -> WPA2 join -> DHCP
wifi on                # 直前の資格情報、またはビルド時の wifi.conf を再利用
wifi scan              # 無線を起こして近傍 AP を一覧
wifi status            # 接続状態・SSID・IP
wifi off               # 無線停止
wifi-invest            # 接続できないときの診断
```

wifi on 成功時は wifi: CONNECTED IP=... で終わる。画面右下に WiFi の電波アイコンと、その下に SSID・自 IP を表示する。資格情報はビルド時に.gitignore 済みの wifi.conf からカーネルへ埋め込まれる（パスワードはコミットしない）。

9.2 メッシュネットワーク（MANET ad-hoc）

複数の Pi 5（および Pi 4 / Pi 3）Xinu ノードをアクセスポイント無しで直接つなぎ、ピアツーピアのメッシュを組める。802.11 の IBSS（ad-hoc）モード + オンデマンドの AODV マルチホップルーティングを使う。

```
wifi adhoc <cell-ssid> [ch] [node] # ad-hoc セルに 10.0.0.<node> で参加
wifi aodv <a.b.c.d>                # マルチホップ経路をオンデマンド探索
```

- wifi adhoc mesh1 6 1 は指定セル（BSSID 02:4d:41:4e:45:54 = “MANET”）のチャンネル 6（既定）で無線を IBSS モードで起こし、静的 IP 10.0.0.1（[node] 番号、既定 1）を取る。以降 AODV 中継が自動で動く。
- 全ノードで同じセル SSID・同じチャンネルを使い、[node] 番号は重複させない（10.0.0.<node>/24）。電波の届く範囲のノードは 10.0.0.x で直接通信できる。
- 直接届かない宛先には wifi aodv 10.0.0.3 で RREQ をブロードキャスト（UDP 654）し、中間ノードが中継、宛先が RREP を返してマルチホップ経路を確立する（例 AODV route: 10.0.0.3 via 10.0.0.2, 2 hop）。先に wifi adhoc で IP を取得しておくこと。

ad-hoc/AODV はインフラ（wifi on）モードとは独立で、再起動後は復元しないため、各ノードで

wifi adhoc を再実行する。

10 USB（マウス・キーボード）

USB-A ポートは RP1 内の 2 つの DWC3/xHCI コントローラに接続される。ブート時のフォールトを避けるため、列挙は起動数秒後に自動で行われる（あるいは /usb/... エンドポイントから手動駆動）。ブートマウスはカーソル移動に、キーボードはシェル入力に用いられる。

11 付録

11.1 ビルドターゲット一覧

ターゲット	生成物
make pi5	kernel_2712.img（フル OS）
make pi5-osmin	kernel_min.img（OS1）
make pi5-os2	kernel_os2.img（OS2）
make pi5-os3	kernel_os3.img（OS3）

11.2 トラブルシューティング

- **HTTP が応答しない**：起動直後はネットワーク初期化前。十数秒待ってから再試行する。kexec 後はネットワークが落ちるため、電源再投入で USB のフル OS に戻る。
- **kexec で「too large」と出る**：4MB 以上のイメージ（Linux カーネル等）はチェーンロードできない。ベアメタル Xinu のイメージを指定すること。
- **/microsd が空**：スロットに microSD が入っているか確認。sdtest で初期化状況を確認できる。
- **画面が出ない**：HDMI フレームバッファのメールボックス初期化に失敗した場合、コンソールは UART のみとなる。

11.3 安全上の注意

- WiFi のパスワードはソースリポジトリにコミットしない（ビルド時に.gitignore 済みファイルから埋め込む運用）。
- kexec は RAM 上の操作であり、不正なイメージでも電源再投入で復帰する（フラッシュ破壊のリスクはない）。